

1. Diseño de las estructuras

En el presente proyecto se implementó un sistema de gestión para un supermercado utilizando múltiples estructuras de datos especializadas, cada una optimizada para un tipo específico de operación. El núcleo del sistema es la clase **SistemaSupermercado**, la cual se encarga de administrar e integrar todas las estructuras utilizadas.

Las estructuras implementadas son:

- Lista Enlazada
 - Permite almacenar colecciones dinámicas de productos.
 - Facilita recorridos completos del sistema.
- Árbol AVL
 - Organiza los productos por nombre.
 - Permite búsquedas eficientes manteniendo el árbol balanceado.
- Árbol B
 - Organiza los productos según su fecha de caducidad.
 - Permite consultas por rango de fechas.
- Árbol B+
 - Agrupa los productos por categoría.
 - Permite recorridos secuenciales eficientes mediante hojas enlazadas.
- Tabla Hash
 - Permite acceso directo a productos mediante una clave única.
 - Optimiza búsquedas en tiempo constante promedio.
- Grafo de Sucursales
 - Representa las conexiones entre distintas sucursales.
 - Permite gestionar rutas y transferencias de productos.
- Cola
 - Permite gestionar procesos en orden FIFO.
 - Se utiliza para el manejo de solicitudes o transferencias.
- Pila
 - Permite gestionar operaciones tipo LIFO.
 - Se utiliza para control de acciones como rollback.

Cada estructura cumple un rol específico dentro del sistema, evitando sobrecargar una sola estructura con múltiples responsabilidades y mejorando la eficiencia general.

2. Justificación de Big-O

Cada estructura fue seleccionada en función de su complejidad computacional y del tipo de operación que optimiza dentro del sistema.

Operación	Estructura	Complejidad
Búsqueda por nombre	Árbol AVL	$O(\log n)$
Inserción general	Lista Enlazada	$O(n)$
Búsqueda por fecha	Árbol B	$O(\log n)$
Búsqueda por categoría	Árbol B+	$O(\log n)$
Recorrido por categoría	Árbol B+	$O(n)$
Acceso por clave	Tabla Hash	$O(1)^*$

Justificación:

- El Árbol AVL mantiene el balance del árbol, garantizando búsquedas eficientes en tiempo logarítmico.
- La Tabla Hash permite acceso casi inmediato mediante una función hash bien definida.
- El Árbol B reduce la altura del árbol, siendo ideal para manejar grandes volúmenes de datos.
- El Árbol B+ optimiza recorridos secuenciales gracias al enlace entre hojas.
- La Lista Enlazada funciona como estructura base flexible para almacenamiento dinámico.
- El Grafo permite modelar relaciones complejas entre sucursales que otras estructuras no pueden representar.

3. Metodología de medición

Para evaluar el rendimiento del sistema se realizaron pruebas experimentales utilizando conjuntos de datos con distintos tamaños.

Procedimiento:

1. Se generaron e insertaron productos dentro de todas las estructuras.
2. Se ejecutaron operaciones de búsqueda, inserción y recorrido.
3. Se midió el tiempo de ejecución utilizando funciones de tiempo en Java.

Ejemplo de medición:

```
long inicio = System.nanoTime();
```

```
// operación a medir
```

```
long fin = System.nanoTime();  
long tiempo = fin - inicio;
```

Se realizaron múltiples ejecuciones para obtener resultados representativos y reducir variaciones.

4. Resultados

Ejemplo de resultados obtenidos:

Estructura	Tiempo (ms)
Lista Enlazada	15
AVL	20
Hash	5

Análisis:

- En conjuntos de datos pequeños, algunas estructuras simples como la lista pueden mostrar tiempos competitivos debido a su bajo costo de implementación.
- Sin embargo, a medida que el volumen de datos crece, las estructuras como AVL, B y B+ presentan un mejor rendimiento debido a su complejidad logarítmica.
- La tabla hash mantiene el mejor rendimiento en accesos directos.

5. Comparación entre estructuras

Las estructuras implementadas presentan diferentes ventajas dependiendo del tipo de operación:

- AVL: eficiente para búsquedas ordenadas por nombre.
- Árbol B: ideal para consultas por rango (fechas).
- Árbol B+: eficiente para agrupaciones y recorridos secuenciales.
- Hash: mejor opción para acceso directo por clave.
- Lista: útil como estructura base, pero limitada en búsquedas grandes.
- Grafo: permite representar relaciones entre sucursales.

El uso combinado permite aprovechar las ventajas de cada una.

6. Conclusiones

- El uso de múltiples estructuras permitió optimizar diferentes tipos de consultas dentro del sistema.
- La tabla hash resultó ser la estructura más eficiente para accesos directos.
- El árbol AVL garantiza eficiencia en búsquedas por nombre.
- Los árboles B y B+ son adecuados para manejar grandes volúmenes de datos y consultas específicas.
- Las estructuras lineales como listas, colas y pilas cumplen funciones auxiliares importantes.